

# Practical Week-10

**Task 1:** Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
lenovo@Manya:~$ mkdir -p ~/OSSP/InodeDemo
lenovo@Manya:~$ cd ~/OSSP/InodeDemo
lenovo@Manya:~/OSSP/InodeDemo$ pwd
/home/lenovo/OSSP/InodeDemo
lenovo@Manya:~/OSSP/InodeDemo$ echo "This is the original file." > original.txt
lenovo@Manya:~/OSSP/InodeDemo$ cat original.txt
This is the original file.
lenovo@Manya:~/OSSP/InodeDemo$ ls -li original.txt
26092 original.txt
lenovo@Manya:~/OSSP/InodeDemo$ stat original.txt
  File: original.txt
  size: 27          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 26092    Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/lenovo)   Gid: ( 1000/lenovo)
Access: 2026-09-24 15:04:06.972130122 +0000
Modify: 2026-09-24 15:03:58.584131502 +0000
Change: 2026-09-24 15:03:58.584131502 +0000
 Birth: 2026-09-24 15:03:58.584131502 +0000
lenovo@Manya:~/OSSP/InodeDemo$ find . -inum $(ls -li original.txt | awk '{print $1}')
./original.txt
lenovo@Manya:~/OSSP/InodeDemo$ ln original.txt hardlink.txt
lenovo@Manya:~/OSSP/InodeDemo$ ls -l
total 8
-rw-r--r-- 2 lenovo lenovo 27 Sep 24 15:03 hardlink.txt
-rw-r--r-- 2 lenovo lenovo 27 Sep 24 15:03 original.txt
lenovo@Manya:~/OSSP/InodeDemo$ ls -li original.txt hardlink.txt
26092 hardlink.txt 26092 original.txt
lenovo@Manya:~/OSSP/InodeDemo$ stat original.txt
  File: original.txt
  size: 27          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 26092    Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/lenovo)   Gid: ( 1000/lenovo)
Access: 2026-09-24 15:04:06.972130122 +0000
Modify: 2026-09-24 15:03:58.584131502 +0000
Change: 2026-09-24 15:05:32.356117045 +0000
 Birth: 2026-09-24 15:03:58.584131502 +0000
lenovo@Manya:~/OSSP/InodeDemo$ stat hardlink.txt
  File: hardlink.txt
  size: 27          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 26092    Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/lenovo)   Gid: ( 1000/lenovo)
Access: 2026-09-24 15:04:06.972130122 +0000
Modify: 2026-09-24 15:03:58.584131502 +0000
Change: 2026-09-24 15:05:32.356117045 +0000
 Birth: 2026-09-24 15:03:58.584131502 +0000
```

```

Device: 8:48 Inode: 26092 Links: 2
Access: (0644/-rw-r--r--) Uid: ( 1000/lenovo) Gid: ( 1000/lenovo)
Access: 2026-09-24 15:04:06.972130122 +0000
Modify: 2026-09-24 15:03:58.584131502 +0000
Change: 2026-09-24 15:05:32.356117045 +0000
 Birth: 2026-09-24 15:03:58.584131502 +0000
lenovo@Manya:~/OSSP/InodeDemo$ echo "Data added through hard link." >> hardlink.txt
lenovo@Manya:~/OSSP/InodeDemo$ cat original.txt
This is the original file.
Data added through hard link.
lenovo@Manya:~/OSSP/InodeDemo$ ln -s original.txt symlink.txt
lenovo@Manya:~/OSSP/InodeDemo$ ls -l
total 8
-rw-r--r-- 2 lenovo lenovo 57 Sep 24 15:08 hardlink.txt
-rw-r--r-- 2 lenovo lenovo 57 Sep 24 15:08 original.txt
lrwxrwxrwx 1 lenovo lenovo 12 Sep 24 15:08 symlink.txt -> original.txt
lenovo@Manya:~/OSSP/InodeDemo$ ls -l original.txt hardlink.txt symlink.txt
26092 hardlink.txt 26092 original.txt 26093 symlink.txt
lenovo@Manya:~/OSSP/InodeDemo$ ls -li original.txt
26092 original.txt
lenovo@Manya:~/OSSP/InodeDemo$ find . -inum 26092
./hardlink.txt
./original.txt
lenovo@Manya:~/OSSP/InodeDemo$ rm original.txt
lenovo@Manya:~/OSSP/InodeDemo$ ls -l
total 4
-rw-r--r-- 1 lenovo lenovo 57 Sep 24 15:08 hardlink.txt
lrwxrwxrwx 1 lenovo lenovo 12 Sep 24 15:08 symlink.txt -> original.txt
lenovo@Manya:~/OSSP/InodeDemo$ cat hardlink.txt
This is the original file.
Data added through hard link.
lenovo@Manya:~/OSSP/InodeDemo$ |

```

**Task 2:** Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```

lenovo@Manya:~$ cd ~/OSSP
lenovo@Manya:~/OSSP$ mkdir -p MMapDemo
lenovo@Manya:~/OSSP$ cd MMapDemo
lenovo@Manya:~/OSSP/MMapDemo$ echo "Hello from memory mapped file I/O." > input.txt
lenovo@Manya:~/OSSP/MMapDemo$ cat input.txt
Hello from memory mapped file I/O.
lenovo@Manya:~/OSSP/MMapDemo$ nano mmap_file.c
lenovo@Manya:~/OSSP/MMapDemo$ gcc mmap_file.c -o mmap_file
lenovo@Manya:~/OSSP/MMapDemo$ ./mmap_file
File size: 35 bytes
Original content:
Hello from memory mapped file I/O.

File modified successfully using mmap().
lenovo@Manya:~/OSSP/MMapDemo$ cat input.txt
Modified using mmap file I/O.
I/O.
lenovo@Manya:~/OSSP/MMapDemo$ |

```

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    struct stat file_info;
    char *mapped;

    // Open the file
    fd = open("input.txt", O_RDWR);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    // Get file size
    if (fstat(fd, &file_info) < 0)
    {
        perror("fstat");
        close(fd);
        return 1;
    }

    printf("File size: %ld bytes\n", (long)file_info.st_size);

    // Map file into memory
    mapped = mmap(NULL,
                  file_info.st_size,
                  PROT_READ | PROT_WRITE,
                  MAP_SHARED,
                  fd,
                  0);

    if (mapped == MAP_FAILED)
    {
        perror("mmap");
        close(fd);
        return 1;
    }

    // Read data through memory
    printf("Original content:\n%s\n", mapped);

    // Modify file through mapped memory
```

```

// Modify file through mapped memory
const char *new_text = "Modified using mmap file I/O.\n";

if (strlen(new_text) <= (size_t)file_info.st_size)
{
    memcpy(mapped, new_text, strlen(new_text));

    // Synchronize changes with file
    if (msync(mapped, file_info.st_size, MS_SYNC) < 0)
    {
        perror("msync");
    }
}
else
{
    printf("New text is larger than the existing file.\n");
}

// Unmap memory
munmap(mapped, file_info.st_size);

close(fd);

printf("File modified successfully using mmap().\n");

return 0;
}

```

```

lenovo@Manya:~/OSSP/MMapDemo$ nano traditional_io.c
lenovo@Manya:~/OSSP/MMapDemo$ gcc traditional_io.c -o traditional_io
lenovo@Manya:~/OSSP/MMapDemo$ ./traditional_io
File copied using read() and write().
lenovo@Manya:~/OSSP/MMapDemo$ cat output_readwrite.txt
Modified using mmap file I/O.
T/n
lenovo@Manya:~/OSSP/MMapDemo$ |

```

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    int output_fd;
    char buffer[1024];
    ssize_t bytesRead;

    fd = open("input.txt", O_RDONLY);

    if (fd < 0)
    {
        perror("open input");
        return 1;
    }

    output_fd = open("output_readwrite.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0)
    {
        perror("open output");
        close(fd);
        return 1;
    }

    while ((bytesRead = read(fd, buffer, sizeof(buffer))) > 0)
    {
        write(output_fd, buffer, bytesRead);
    }

    if (bytesRead < 0)
    {
        perror("read");
    }

    close(fd);
    close(output_fd);

    printf("File copied using read() and write().\n");

    return 0;
}
```